

Universidade Federal do Pará
Faculdade de Sistemas de Informação

FasiTech

Fundamentos Científicos do Sistema

Módulo de Recuperação com Geração Aumentada (RAG)
Módulo de Extração Inteligente de Documentos (ACC)

Sumário

1	Introdução	3
2	Módulo RAG — Diretor Virtual	4
2.1	Fundamentos: Retrieval-Augmented Generation	4
2.1.1	Recuperação Densa com Bi-Encoders	4
2.1.2	Busca Híbrida: Semântica + BM25	5
2.2	Estratégias de Chunking	6
2.2.1	Chunking Básico com Sobreposição	6
2.2.2	Chunking Contextual de Seção (Melhoria 1)	7
2.2.3	Chunking Hierárquico com <i>Breadcrumb</i> de Seção (Melhoria 3)	9
2.2.4	Chunking Pai-Filho (<i>Small-to-Big</i>)	10
2.3	Cache Semântico	10
2.3.1	Motivação e Arquitetura	10
2.3.2	Ciclo de Vida das Entradas	11
2.4	Filtragem de Domínio	11
2.5	Detecção de Ambiguidade Temporal (Melhoria 2)	12
2.6	Expansão de Consulta por Siglas	13
2.7	Reranking com Cross-Encoder (Melhoria 4)	14
2.8	Visão Geral do Pipeline RAG	16
3	Arquitetura Omnichannel de Atendimento	17
3.1	Motivação e Visão Geral	17
3.2	Canais e <i>Inboxes</i>	18
3.3	Roteamento por <i>Webhook</i> : Chatwoot → FastAPI	19
3.4	Sessão por Conversa	21
3.5	Máquina de Estados do Diretor Virtual	22
3.6	Endpoints Principais	22
3.7	Escalação para Atendimento Humano	23
3.8	Extensibilidade Multi-Canal: o Caso do WhatsApp	24
3.9	Considerações de Segurança	25

4	Módulo ACC — Extração Inteligente de Documentos	26
4.1	Contexto e Desafio	26
4.2	Fundamentos: Document AI e Multimodal LLMs	26
4.2.1	Modelos Multimodais	26
4.2.2	Processamento de PDFs com pdf2image e Poppler	27
4.3	Arquitetura do Módulo ACC	28
4.4	Prompt Engineering para Extração Estruturada	28
4.4.1	Extração do Total por Expressão Regular	29
4.5	Análise de Confiabilidade e Limitações	29
4.5.1	Fontes de Erro	29
4.5.2	Escalabilidade	29
5	Infraestrutura e Integração	30
5.1	Banco de Dados Vetorial: LanceDB	30
5.2	Modelos de Linguagem e Embedder	30
5.3	Padrão de Projeto: Singleton com Inicialização Lazy	31
6	Histórico de Estratégias e Benchmarking	32
7	Gestão de Funcionários e Roteamento Dinâmico de Notificações	34
7.1	Contexto e Motivação	34
7.2	Modelo de Dados: a Entidade <code>Funcionario</code>	35
7.3	Seletores Dinâmicos nos Formulários	36
7.4	Roteamento de Notificações por Papel	37
8	Geração Automática de Ata de Defesa de TCC	39
8.1	Contexto e Motivação	39
8.2	Arquitetura da Solução	40
8.2.1	Recuperação de Dados Enriquecidos	40
8.2.2	Composição do Documento com <code>python-docx</code>	41
8.2.3	Normalização de Nomes no Arquivo	41
8.2.4	Envio por E-mail com Instruções SIPAC	42
8.3	Interface de Usuário	42
8.4	Tratamento de Erros e Validações	43
9	Trabalhos Futuros	44

Capítulo 1

Introdução

O sistema FasiTech é uma plataforma web institucional desenvolvida para centralizar fluxos acadêmicos da Faculdade de Sistemas de Informação (FASI) da UFPA. Entre seus módulos, dois se apoiam diretamente em técnicas de Inteligência Artificial:

1. **Diretor Virtual** — assistente conversacional baseado em *Retrieval-Augmented Generation* (RAG) que responde perguntas sobre regulamentos, cronogramas e documentos acadêmicos.
2. **Módulo ACC** — extrator automático de cargas horárias de certificados enviados em formato PDF, usando Modelos de Linguagem Multimodais (*Multimodal Large Language Models*).

Este documento registra a ciência aplicada em cada componente, servindo de referência para manutenção, evolução e publicação futura dos resultados.

Capítulo 2

Módulo RAG — Diretor Virtual

2.1 Fundamentos: Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) é uma arquitetura que combina recuperação de documentos com geração de linguagem natural. A formulação original foi proposta por Lewis et al. (2020): dado um modelo generativo $p_\theta(y \mid x)$ e um índice de documentos $\mathcal{D}$, o modelo recupera os documentos mais relevantes $\mathbf{z} = \text{Top-}k(q, \mathcal{D})$ e condiciona a geração sobre eles:

$$p(y \mid x) = \sum_{z \in \mathbf{z}} p_\eta(z \mid x) p_\theta(y \mid x, z) \quad (2.1)$$

onde $p_\eta(z \mid x)$ é a probabilidade do documento z ser relevante para a consulta x , calculada via busca densa, e p_θ é o LLM gerador.

Por que RAG e não fine-tuning?

Fine-tuning parametriza o conhecimento no modelo, tornando atualizações custosas e sem rastreabilidade de fonte. RAG externaliza o conhecimento nos documentos, permitindo atualização imediata sem retreinamento e garantindo que respostas sejam fundamentadas em trechos verificáveis.

2.1.1 Recuperação Densa com Bi-Encoders

A recuperação densa (DR — *Dense Retrieval*) representa consultas e documentos como vetores densos em um espaço semântico compartilhado. O trabalho seminal de Karpukhin et al. (2020) (DPR — *Dense Passage Retrieval*) demonstrou que bi-encoders treinados por aprendizado contrastivo superam a busca esparsa BM25 em domínios factuais.

A similaridade entre a consulta q e um documento d é calculada por:

$$\text{sim}(q, d) = \cos(E_Q(q), E_D(d)) = \frac{E_Q(q) \cdot E_D(d)}{\|E_Q(q)\| \|E_D(d)\|} \quad (2.2)$$

No sistema FasiTech, o embedder é o **Ollama Embedder** (`nomic-embed-text` (NUSBAUM et al., 2024)), executado localmente via servidor Ollama (OLLAMA, 2023) (`OLLAMA_HOST`, padrão `localhost:11434`), produzindo vetores de 768 dimensões. Todos os vetores são normalizados pelo serviço (norma $L2 = 1$) antes de serem armazenados no LanceDB, permitindo que a similaridade cosseno seja computada como produto escalar simples — $\text{sim}(q, d) = E(q) \cdot E(d)$.

Prefixos de Tarefa (Melhoria 5)

Estratégia Anterior (preservada para benchmarking)

A versão inicial gerava embeddings chamando o mesmo método (`_get_question_embedding`) tanto para os chunks indexados quanto para a consulta, *sem nenhum prefixo de tarefa*.

Problema Observado

O `nomic-embed-text` é treinado para *recuperação assimétrica* e **exige** prefixos de instrução: `search_document`: para o texto indexado e `search_query`: para a consulta (NUSSBAUM et al., 2024). Omiti-los faz consulta e documento serem projetados como se fossem do mesmo tipo, degradando mensuravelmente o *recall* — efeito agravado em consultas curtas e factuais como “ementa da disciplina X”.

A versão atual aplica o prefixo correto conforme o papel do texto (`input_type` $\in \{\text{query}, \text{document}\}$), apenas quando o embedder é o `nomic`. Os documentos são reindexados como `search_document`: `<chunk>` e a consulta é embeddada como `search_query`: `<pergunta>`, restaurando o regime assimétrico para o qual o modelo foi treinado.

2.1.2 Busca Híbrida: Semântica + BM25

Busca híbrida combina recuperação densa (semântica) com recuperação esparsa baseada em frequência de termos (ROBERTSON et al., 1994). O score BM25 para um documento d dado uma consulta q com termos $\{q_i\}$ é:

$$\text{BM25}(d, q) = \sum_i \text{IDF}(q_i) \cdot \frac{f(q_i, d) (k_1 + 1)}{f(q_i, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgl}}\right)} \quad (2.3)$$

onde k_1 e b são hiperparâmetros de saturação de frequência e normalização por comprimento do documento, respectivamente.

O banco vetorial **LanceDB** utilizado no FasiTech configura `SearchType.hybrid`, combinando os dois rankings via *Reciprocal Rank Fusion* (RRF) (CORMACK; CLARKE; BUETTER, 2009):

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + r(d)} \quad (2.4)$$

onde $r(d)$ é a posição do documento no ranking r e $k = 60$ é constante.

Estratégia Anterior (preservada para benchmarking)

Apesar de a documentação referenciar RRF, a primeira implementação de `_retrieve_context` não aplicava a Equação (2.4). Ela executava a busca semântica, recuperava `top_k=10` chunks e, em seguida, simplesmente *inseria no início da lista* os acertos de `_keyword_search` (*insert-at-front*), sem fundir as posições dos dois rankings.

Problema Observado

Duas falhas decorriam dessa fusão ingênua:

1. **Dominância de um recuperador** — correspondências exatas de palavra-chave eram sempre promovidas ao topo, mesmo quando menos relevantes que bons resultados semânticos (e vice-versa).
2. **Recall insuficiente para discriminadores fracos** — a busca densa não distingue bem “I” de “II”. Para a consulta “*ementa de Estrutura de Dados II*”, o chunk correto **não aparecia no top-20 semântico**; era encontrado apenas pela busca esparsa (posição 10). Como o `top_k` era pequeno, o chunk ficava de fora do conjunto entregue ao gerador, que então respondia “não encontrei” para uma disciplina que *existe* no PPC.

A versão atual implementa o RRF de fato (`_rrf_fuse`, $k = 60$), fundindo o ranking denso e o ranking esparsa (`_keyword_search`) por suas *posições*. Além disso, cada recuperador contribui com um *fan-out* ampliado (`RETRIEVER_FANOUT=30`) e a lista fundida alimenta um *pool* de candidatos maior (`RERANK_CANDIDATE_POOL=40`) antes do *reranking* (Seção 2.7). O *pool* amplo é essencial: garante que acertos encontrados *apenas* pela busca esparsa (como “Estrutura de Dados II”) cheguem ao reordenador, em vez de serem cortados prematuramente.

2.2 Estratégias de Chunking

2.2.1 Chunking Básico com Sobreposição

A fragmentação de documentos (*chunking*) é etapa crítica no pipeline RAG. O método original no FasiTech divide textos em blocos de 800 caracteres com sobreposição (*overlap*) de

150 caracteres, respeitando quebras de parágrafo. Esta abordagem, denominada *fixed-size chunking with overlap*, minimiza a perda de contexto nas fronteiras dos chunks.

2.2.2 Chunking Contextual de Seção (Melhoria 1)

A limitação do chunking simples é a perda do contexto hierárquico do documento. Quando um chunk pertence à seção “Período Letivo 2026.2”, o título dessa seção pode estar em um chunk anterior, tornando a recuperação dependente de que o usuário mencione explicitamente “2026.2” na consulta.

Inspirado no *Contextual Retrieval* proposto pela Anthropic (ANTHROPIC, 2024) e no trabalho de Shi et al. (2023) sobre prepend de contexto para melhoria de recuperação, o FasiTech implementa o método `_split_with_section_context`:

1. O documento Markdown é percorrido linha a linha.
2. Cabeçalhos (`#`, `##`, `###`, `####`) são detectados via expressão regular `^(#{1,4})\s+(.+)` e armazenados como `section_header` corrente.
3. Cada chunk gerado pelo método base (`_split_into_chunks`) recebe um *prefixo contextual*:

[Documento: X | Seção: Y]\n\n<chunk>

4. O par (`section_header`, `chunk_contextualizado`) é retornado; o campo `section` é persistido nos metadados do LanceDB.
5. O chunk contextualizado — não o texto bruto — é embeddado e indexado.

```

1 lines = text.split("\n")
2 heading_re = re.compile(r"^(#{1,4})\s+(.+)")
3
4 # Agrupa linhas por secao
5 sections: List[tuple[str, List[str]]] = []
6 current_header, current_lines = "", []
7 for line in lines:
8     m = heading_re.match(line)
9     if m:
10         if current_lines:
11             sections.append((current_header, current_lines))
12             current_header = m.group(2).strip()
13             current_lines = []
14         else:
15             current_lines.append(line)

```



```

16
17 # Gera chunks contextualizados
18 result: List[tuple[str, str]] = []
19 for sec_header, sec_lines in sections:
20     block = "\n".join(sec_lines).strip()
21     raw_chunks = ChatbotService._split_into_chunks(block,
22         chunk_size, overlap)
23     for chunk in raw_chunks:
24         prefix = f"[Documento: {doc_name} | Secao: {sec_header}]\n\n"
25         contextualized = (prefix + chunk) if sec_header else
            chunk
        result.append((sec_header, contextualized))

```

Listing 2.1: Núcleo de `_split_with_section_context` (`rag_ppc.py`).

O efeito é que o embedding do chunk carrega informação da seção pai, aumentando a similaridade com consultas que referenciam aquela seção, mesmo que o chunk em si não a mencione.

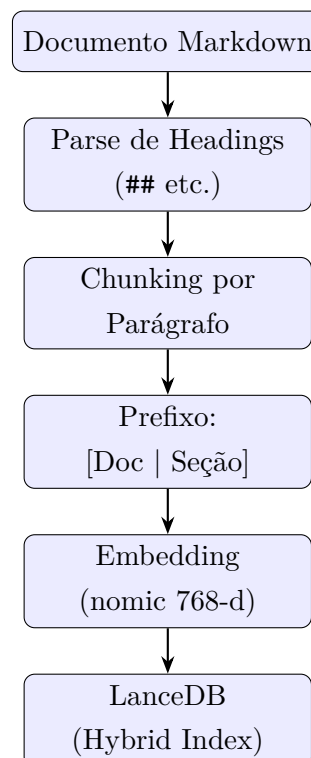


Figura 2.1: Pipeline de indexação com chunking contextual de seção.

2.2.3 Chunking Hierárquico com *Breadcrumb* de Seção (Melhoria 3)

Estratégia Anterior (preservada para benchmarking)

A Melhoria 1 (Seção 2.2.2) detectava *qualquer* cabeçalho Markdown via `^(#{1,4})\s+(.+)` e o tomava como a seção corrente do chunk. O cabeçalho mais recente substituía o anterior, independentemente de seu nível ou natureza.

Problema Observado

No PCC.md, cada disciplina segue o padrão:

```
## 6.1.19.  ESTRUTURA DE DADOS II (60 horas-aula)
```

```
## Ementa:
```

```
Árvores e percursos... Tabela de Hash.
```

Como `## Ementa:` é um cabeçalho *do mesmo nível* da disciplina, ele **substituía** o cabeçalho da disciplina. Resultado: o texto da ementa era indexado sob a seção genérica “Ementa:” e o *nome da disciplina era descartado do chunk*. Todas as ementas do curso compartilhavam o mesmo prefixo [... | Seção: Ementa:]. A consequência prática: *Estrutura de Dados I* ainda era recuperável por acaso — seu corpo repete as palavras “estrutura” e “dados”. Já a ementa de *Estrutura de Dados II* (apenas 147 caracteres, sobre “árvores/grafos/hash”) não contém nenhum desses termos; sem o nome da disciplina no chunk, tornava-se **não-recuperável**, e o sistema afirmava que a disciplina “não existe”. O defeito afetava *toda* disciplina cujo corpo da ementa não repetisse o próprio nome (ex.: *Cálculo Computacional II*).

A versão atual constrói um *breadcrumb* hierárquico de cabeçalho. Um cabeçalho *numerado* (que inicia com dígito, ex.: 6.1.19.) inaugura uma nova seção *forte* (a disciplina). Cabeçalhos *genéricos* — terminados em “:” ou pertencentes a um conjunto conhecido (Ementa, Bibliografia básica, Bibliografia complementar, Objetivos, ...) — **não** substituem a disciplina: são anexados como subseção. O prefixo resultante preserva o nome da disciplina junto ao texto:

```
[Documento: PCC.md | Seção: 6.1.19.  ESTRUTURA DE DADOS II (60 horas-aula) >
Ementa:]
```

```
1 @staticmethod
2 def _is_subsection_header(text: str) -> bool:
3     t = text.strip()
4     if not t or t[0].isdigit():      # "6.1.19. ..." => nova
        disciplina
```

```

5         return False
6         norm = t.lower().rstrip(":").strip()
7         return t.endswith(":") or norm in _GENERIC_SUBHEADERS
8
9     # No laço de parsing:
10    if _is_subsection_header(heading) and parent_header:
11        current_header = f"{parent_header} > {heading}"    # 6.1.19.
12        ... > Ementa:
13    else:
14        parent_header = heading                                # nova
15        secao forte
16        current_header = heading

```

Listing 2.2: Decisão de subseção genérica vs. seção forte (`rag_ppc.py`).

Com o nome da disciplina de volta ao chunk, ela passa a ser recuperável tanto pela busca esparsa (`_keyword_search`) quanto pelo *reranker*. **Esta foi a correção da causa-raiz** do defeito relatado em produção.

Por que não usar apenas o nível do cabeçalho?

A hierarquia por nível (`# > ## > ###`) não resolve o caso: no `PCC.md` a disciplina e “Ementa:” estão *ambas* em nível 2 (`##`). Por isso a heurística distingue cabeçalhos *numerados* (estruturais) de *genéricos* (rótulos de conteúdo).

2.2.4 Chunking Pai-Filho (*Small-to-Big*)

Como direção futura, o FasiTech pode adotar o *Parent-Child Chunking* introduzido no LlamaIndex (LLAMAINDEX, 2023): chunks pequenos (200 chars) são indexados para busca precisa, mas o contexto entregue ao LLM é o chunk pai (1 200 chars) que contém o trecho recuperado. Isso reduz o ruído na busca vetorial enquanto mantém contexto suficiente para a geração.

2.3 Cache Semântico

2.3.1 Motivação e Arquitetura

O cache semântico armazena pares (pergunta, resposta) avaliados positivamente e os serve em consultas futuras semanticamente similares, eliminando chamadas ao LLM para perguntas recorrentes. Esta abordagem é referenciada em Bang et al. (2023) e Zhu et al. (2023).

O sistema FasiTech implementa um cache em duas camadas no LanceDB:

Tabela 2.1: Parâmetros do cache semântico do FasiTech.

Parâmetro	Descrição	Valor
Threshold de similaridade	Cosseno mínimo para hit de cache	0,90
Avaliação mínima (<i>trusted</i>)	Média de notas para promover entrada	4,4/5
Avaliações para promoção	Qtd. mínima de feedbacks	2
TTL <i>trusted</i>	Validade de entrada confiável	30 dias
TTL <i>candidate</i>	Validade de entrada candidata	14 dias

2.3.2 Ciclo de Vida das Entradas

Uma entrada percorre o seguinte ciclo:

1. **Candidate** — criada após primeira avaliação positiva ($\text{nota} \geq 3$).
2. **Trusted** — promovida quando $\bar{r} \geq 4,4$ com $n \geq 2$ avaliações.
3. **Servida** — retornada diretamente ao usuário sem chamar o LLM.
4. **Expirada** — removida após TTL ou se nota ≤ 2 na última avaliação.

A função de confiança C combina qualidade e volume:

$$C = \frac{\bar{r} - 1}{4} \cdot \min\left(1, \frac{n}{n_{\min}}\right) \quad (2.5)$$

Adicionalmente, cada entrada armazena o `documents_hash`, garantindo que mudanças nos documentos fontes invalidem automaticamente o cache.

2.4 Filtragem de Domínio

Antes de acionar o RAG, o sistema verifica se a pergunta pertence ao domínio acadêmico para evitar respostas sobre tópicos fora de escopo.

A estratégia é bifásica: (1) detecção de palavras-chave acadêmicas (ex.: “TCC”, “matrícula”, “ACC”) ajusta o limiar de similaridade para mais permissivo ($\tau = 0,50$); (2) na ausência de palavras-chave, um limiar mais estrito é aplicado ($\tau = 0,72$) para evitar que perguntas genéricas em português sejam processadas por similaridade superficial de idioma.

O score combinado usa 40% da melhor similaridade e 60% da média dos três primeiros resultados:

$$S = 0,4 \cdot \text{sim}_{\max} + 0,6 \cdot \bar{\text{sim}}_3 \quad (2.6)$$

2.5 Detecção de Ambiguidade Temporal (Melhoria 2)

Estudos em busca conversacional demonstram que perguntas ambíguas — aquelas que carecem de entidades contextuais necessárias para uma resposta precisa — devem ser respondidas com uma pergunta de esclarecimento antes da recuperação (ALIANNEJADI et al., 2021).

No contexto do FasiTech, a ambiguidade temporal ocorre quando o usuário pergunta sobre prazos sem especificar o período letivo. O método `_needs_temporal_clarification` implementa a seguinte lógica:

$$\text{clarify}(q) = \underbrace{\exists t \in T_{\text{trigger}} : t \in q}_{\text{gatilho temporal}} \wedge \underbrace{\nexists e \in E_{\text{period}} : e \in q}_{\text{ausência de entidade}} \quad (2.7)$$

Os conjuntos são implementados como `frozenset` para busca $O(1)$:

```

1 _TEMPORAL_TRIGGERS: frozenset = frozenset({
2     "periodo", "prazo", "submissao", "entrega", "data", "quando",
3     "limite", "cronograma", "calendario", "vencimento", "ate
4     quando",
5     "deadline", "datas", "prazos", "agenda",
6 })
7
8 _TEMPORAL_ENTITIES: frozenset = frozenset({
9     # Identificadores de periodo letivo: 2024.1 ... 2028.4
10    "2024.1", "2024.2", "2024.3", "2024.4",
11    "2025.1", "2025.2", "2025.3", "2025.4",
12    "2026.1", "2026.2", "2026.3", "2026.4",
13    "2027.1", ..., "2028.4",
14    # Expressoes naturais de periodo
15    "primeiro periodo", "segundo periodo", "semestre atual",
16    "proximo semestre", "este semestre",
17 })
18
19 @staticmethod
20 def _needs_temporal_clarification(question: str) -> bool:
21     q_lower = question.lower()
22     has_trigger = any(t in q_lower for t in _TEMPORAL_TRIGGERS)
23     has_entity = any(e in q_lower for e in _TEMPORAL_ENTITIES)
24     return has_trigger and not has_entity

```

Listing 2.3: Constantes de detecção temporal (`rag_ppc.py`).

Quando $\text{clarify}(q) = \top$, o método `_get_available_periods` varre todos os chunks do LanceDB em busca do padrão `\b(20\d{2}\.[1-4])\b` e retorna os períodos reais indexados:

```

1 def _get_available_periods(self) -> List[str]:
2     period_re = re.compile(r"\b(20\d{2}\.[1-4])\b")
3     found: set = set()
4     df = lancedb.connect(self.db_url).open_table("recipes").
        to_pandas()
5     for row in df.itertuples():
6         p = json.loads(row.payload)
7         for m in period_re.findall(p.get("content", "")):
8             found.add(m)
9     return sorted(found)    # ex.: ['2026.2', '2026.3', '2026.4']

```

Listing 2.4: Descoberta dinâmica de períodos (rag_ppc.py).

A resposta de esclarecimento é formatada em Markdown (renderizado no frontend via `react-markdown + remark-gfm`):

```

1 period_list = ", ".join(available_periods)    # ex.: 2026.2,
        2026.3, 2026.4
2 clarification = (
3     f"Para qual período letivo voce deseja as informacoes?\n\n"
4     f"Períodos disponiveis nos documentos: {period_list}\n\n"
5     f"Por favor, especifique o período (ex.: 2026.2) para que"
6     f"eu "
7     f"possa te dar uma resposta precisa."
8 )

```

Listing 2.5: Mensagem de esclarecimento gerada dinamicamente.

Esta abordagem elimina chamadas desnecessárias ao LLM antes de executar o pipeline RAG completo com contexto insuficiente.

Ganho esperado

Esta abordagem elimina a necessidade de o usuário reformular a pergunta. Em vez de receber uma resposta incompleta ou mesclada entre períodos, o sistema pede esclarecimento na primeira interação, reduzindo o número médio de turnos para resposta de ≈ 2 para 1.

2.6 Expansão de Consulta por Siglas

O método `_expand_query` implementa um mapa de siglas para seus expandidos semânticos (ex.: “ACC” → “Atividade Curricular Complementar atividades complementares ACG

extracurricular”).

Esta técnica é variante do *Query Expansion* estudado em Carpineto e Romano (2012): ao adicionar termos sinônimos à consulta antes do embedding, aumenta-se o recall da busca vetorial sem modificar o banco de vetores.

2.7 Reranking com Cross-Encoder (Melhoria 4)

Estratégia Anterior (preservada para benchmarking)

Nas versões anteriores não havia reordenação: os chunks fundidos seguiam diretamente para a montagem do prompt, na ordem produzida pela recuperação. O reranking constava apenas como “trabalho futuro”.

Problema Observado

Bi-encoders comprimem consulta e documento em vetores *independentes*: a similaridade vetorial aproxima relevância, mas *similaridade* $\neq$ *relevância*. Em consultas com discriminadores fracos (“I” vs. “II”), o ranking denso confunde disciplinas afins. Mesmo com a busca esparsa trazendo o chunk correto ao *pool*, sem um juiz de relevância par-a-par o chunk certo raramente alcançava o topo.

Um *cross-encoder* processa o par (consulta, documento) *conjuntamente* em uma única passagem do *Transformer*, atribuindo um score de relevância direto (KHATTAB; ZAHARIA, 2020):

$$s(q, d) = \text{CrossEncoder}(q \parallel d) \quad (2.8)$$

diferentemente do bi-encoder, que codifica q e d separadamente. O custo é maior (não há pré-cálculo de vetores), por isso o reranker é aplicado apenas sobre o *pool* de candidatos da fusão RRF, e não sobre todo o índice.

No FasiTech, o método `_rerank` reordena os `RERANK_CANDIDATE_POOL= 40` candidatos e mantém os `top_k` melhores. O modelo padrão é o `BAAI/bge-reranker-base` (278M de parâmetros, multilíngue), executado localmente via `sentence-transformers`. O carregamento é *preguiçoso* (singleton) e *fail-open*: se a dependência ou o modelo não estiverem disponíveis, o pipeline mantém a ordenação da fusão RRF, sem interromper o serviço. As variáveis `RERANKER_ENABLED` e `RERANKER_MODEL` controlam o comportamento em produção.

Por que bge-reranker-base e não v2-m3?

O `bge-reranker-v2-m3` (568M) e o `bge-reranker-base` (278M) resolvem igualmente a desambiguação I/II no corpus do FasiTech, mas o `base` usa cerca de metade da RAM e é mais rápido em CPU — decisivo no VPS de produção (2 vCPU / 8 GB), onde a inferência ocorre sem GPU.

Dependência do reranker para a correção do caso “II”

Vale registrar para benchmarking: isoladamente, a fusão RRF *não* coloca “Estrutura de Dados II” no top-8 (a busca esparsa a posiciona por volta da posição 21 no ranking fundido). É a combinação *pool ampliado* + *reranking* que eleva o chunk correto ao topo. Desabilitar o reranker reintroduz o defeito.

2.8 Visão Geral do Pipeline RAG

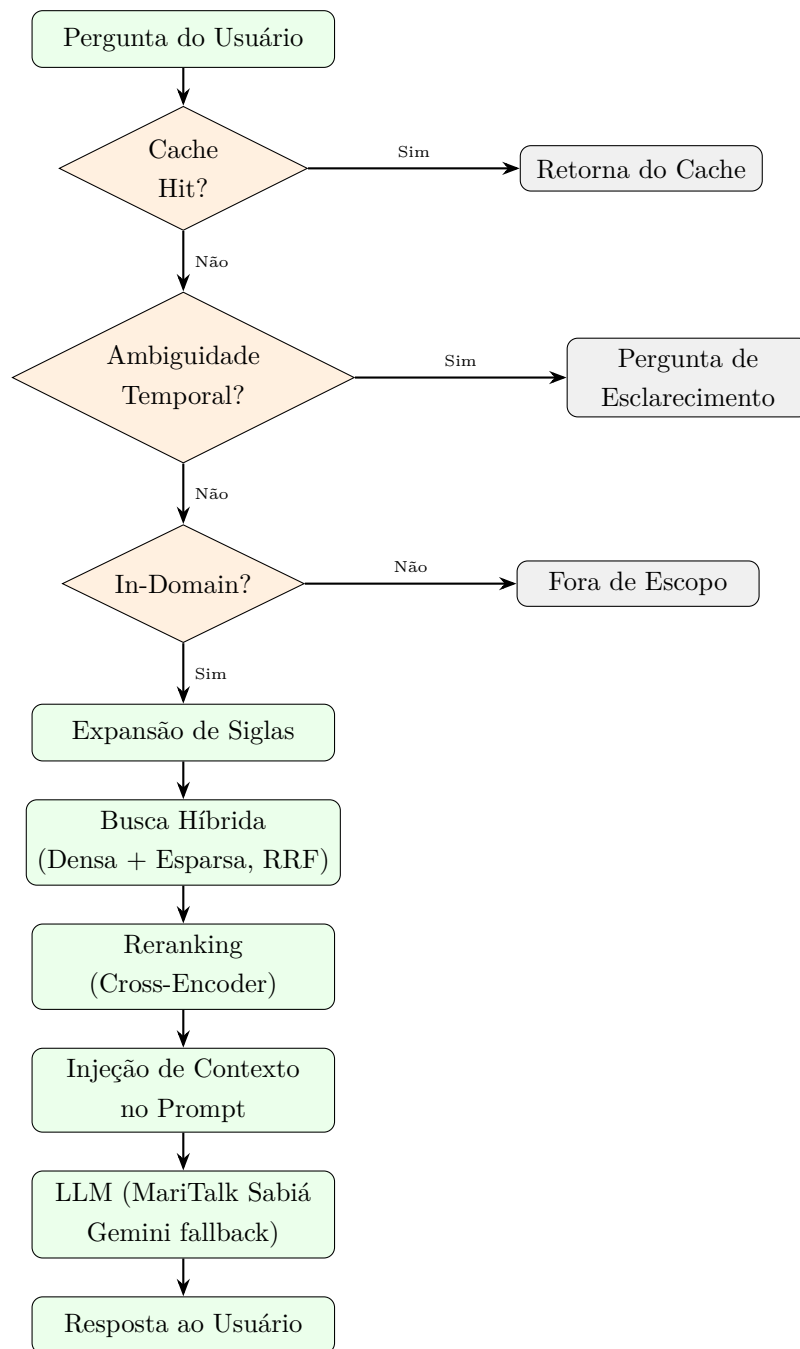


Figura 2.2: Fluxo completo do pipeline RAG do Diretor Virtual.

Capítulo 3

Arquitetura Omnichannel de Atendimento

3.1 Motivação e Visão Geral

O Diretor Virtual, apresentado no capítulo anterior como um pipeline de RAG, precisa alcançar o usuário em múltiplos pontos de contato — o *widget* do portal web e o WhatsApp — sem duplicar a lógica de negócio em cada canal. Essa exigência corresponde ao conceito de **atendimento omnichannel**: uma experiência na qual o histórico, o estado da conversa e as regras de negócio são unificados independentemente do canal de entrada (VERHOEF; KANNAN; INMAN, 2015), em contraste com uma estratégia *multichannel*, em que cada canal opera de forma isolada e com lógica própria.

O FasiTech implementa esse requisito com uma arquitetura de três camadas:

1. **Canais** (*channels*) — pontos de entrada específicos de cada plataforma: o *widget* embutido no portal ( *Website*) e o WhatsApp Business ( *WhatsApp*).
2. **Chatwoot** (CHATWOOT, 2023) (*open-source*) — plataforma *open-source* de atendimento que abstrai os canais em *inboxes* e *conversations*, oferecendo uma única API e um mecanismo de *webhooks* para eventos de mensagem.
3. **Backend FastAPI (FasiTech)** — concentra *toda* a lógica do bot (máquina de estados, RAG, geração de ticket e escalção) em um único serviço, consumido por qualquer canal através do mesmo *webhook*.

Chatwoot como camada de roteamento

Entrega, confirmação de leitura, histórico e painel de atendimento humano já são problemas resolvidos por plataformas de *live chat* dedicadas. Reimplementar isso para cada canal não traria benefício algum — por isso o FasiTech delega esse transporte ao Chatwoot e mantém para si apenas a *lógica de negócio* do bot.

3.2 Canais e *Inboxes*

Cada canal é registrado no Chatwoot como uma *inbox*, associada a um `channel_type` que determina o transporte usado. A Tabela 3.1 lista as *inboxes* da conta FASI e indica quais delas são processadas automaticamente pelo bot.

Inbox	ID	channel_type	Bot ativo?
ChatWeb	5	Channel::WebWidget	Sim — canal de origem deste capítulo
WhatsApp Fasi	3	Channel::Whatsapp	Sim — reaproveita o mesmo <i>webhook</i>
E-mail FASI	4	Channel::Email	Não — fora do escopo do bot conversacional

Tabela 3.1: Inboxes da conta Chatwoot da FASI e participação no fluxo do bot.

A Figura 3.1 ilustra a convergência dos dois canais habilitados para o Chatwoot e, a partir dele, para o *webhook* único consumido pelo backend.

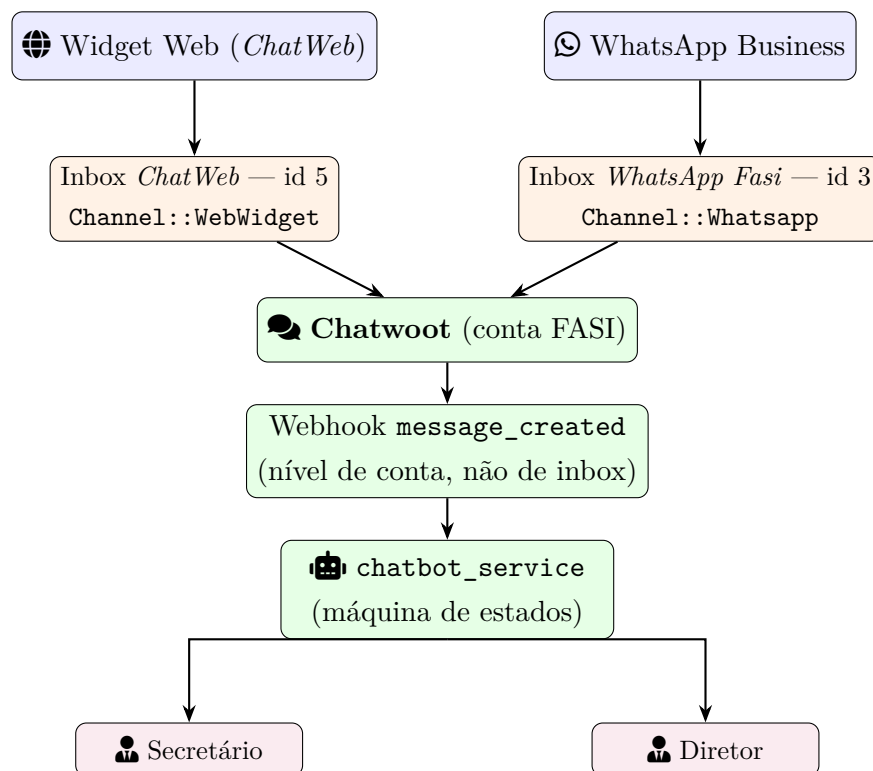


Figura 3.1: Panorama geral: dois canais convergem para o Chatwoot; o *webhook* de conta aciona o backend, que pode escalar a conversa para os times humanos.

Webhooks de conta, não de inbox

No Chatwoot, um *webhook* é assinado no nível da **conta**: uma vez configurado o evento `message_created`, ele dispara para mensagens de *qualquer* inbox daquela conta. Essa característica é o que permite reaproveitar o mesmo *endpoint* para o widget web e para o WhatsApp sem nenhuma configuração adicional no Chatwoot — apenas uma *allowlist* de `inbox_id` no backend (Seção 3.8) decide quais mensagens acionam o bot.

3.3 Roteamento por *Webhook*: Chatwoot → FastAPI

O *endpoint* POST `/api/v1/chatbot/webhook/chatwoot` é o único ponto de entrada do bot para mensagens recebidas via Chatwoot, independente do canal de origem. A Figura 3.2 detalha o processamento de cada requisição recebida.

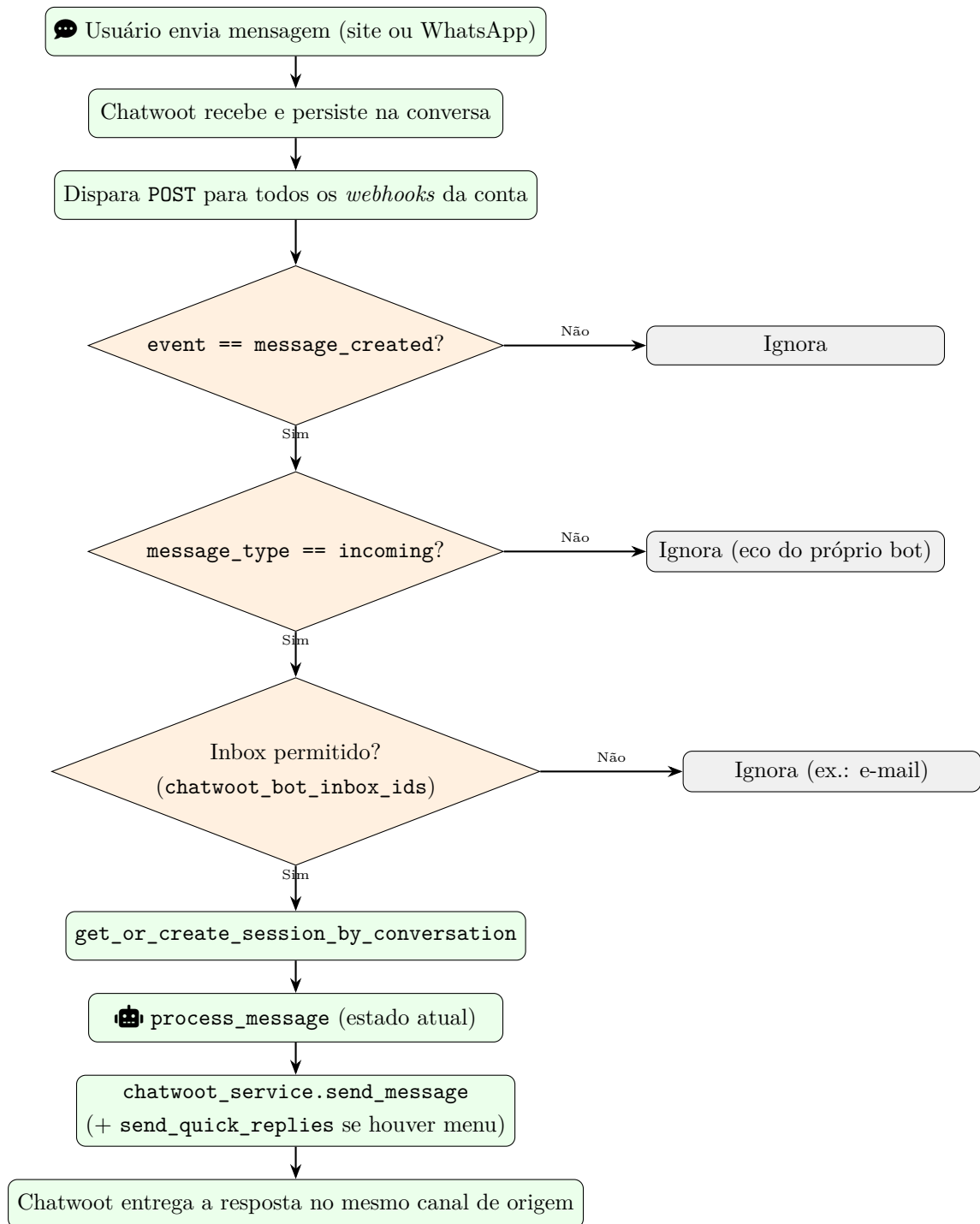


Figura 3.2: Fluxo ponta a ponta de uma mensagem através do webhook `/api/v1/chatbot/webhook/chatwoot`.

O filtro de `message_type` é o mecanismo que impede um laço infinito: como o próprio bot publica suas respostas no Chatwoot via `send_message` (mensagens `outgoing`), sem esse filtro cada resposta do bot geraria um novo evento `message_created` que seria, por sua vez, reprocessado pelo próprio bot.

```
1 if payload.get("event") != "message_created":
```

```

2     return {"ok": True, "skipped": "evento ignorado"}
3
4 if payload.get("message_type") != "incoming":
5     return {"ok": True, "skipped": "nao e mensagem do visitante"}
6
7 inbox_id = conversation.get("inbox_id")
8 if inbox_id and inbox_id not in settings.chatwoot_bot_inbox_ids:
9     return {"ok": True, "skipped": f"inbox {inbox_id} nao e um
    inbox do chatbot"}

```

Listing 3.1: Filtros de entrada do webhook (chatbot_router.py).

3.4 Sessão por Conversa

Diferentemente do fluxo REST legado (POST /chatbot/start e POST /chatbot/message), em que o cliente gerencia um `session_id` próprio, o fluxo omnichannel não tem acesso a esse identificador — o widget nativo do Chatwoot e o WhatsApp não sabem nada sobre a sessão do backend. A função `get_or_create_session_by_conversation` resolve isso indexando a sessão pelo `conversation_id` do Chatwoot, estável durante toda a conversa, em qualquer canal:

```

1 _sessions_by_conversation: dict[int, ChatSession] = {}
2
3 def get_or_create_session_by_conversation(conversation_id: int)
4     -> tuple[ChatSession, bool]:
5     existing = _sessions_by_conversation.get(conversation_id)
6     if existing:
7         existing.touch()
8         return existing, False
9     sess = ChatSession(session_id=str(uuid.uuid4()),
10                        chatwoot_conversation_id=conversation_id)
11     _sessions_by_conversation[conversation_id] = sess
12     return sess, True

```

Listing 3.2: Sessão endereçada por conversa (chatbot_service.py).

Quando a sessão é criada pela primeira vez, a primeira mensagem do usuário *não* é interpretada como dado de cadastro — o backend responde imediatamente com a saudação e o pedido de matrícula, replicando o comportamento do fluxo REST legado (onde a primeira chamada a /chatbot/start também ignora qualquer texto e retorna a saudação). Na prática, isso é o que faz o fluxo de perguntas começar assim que o chat é aberto: seja qual for o conteúdo da primeira mensagem, ela já inicia a sessão e recebe a saudação como

resposta.

3.5 Máquina de Estados do Diretor Virtual

A lógica conversacional é implementada como uma máquina de estados finita (**ChatState**), completamente desacoplada do canal de transporte — a mesma máquina atende ao widget web, ao WhatsApp e ao fluxo REST legado. A Figura 3.3 apresenta as transições.

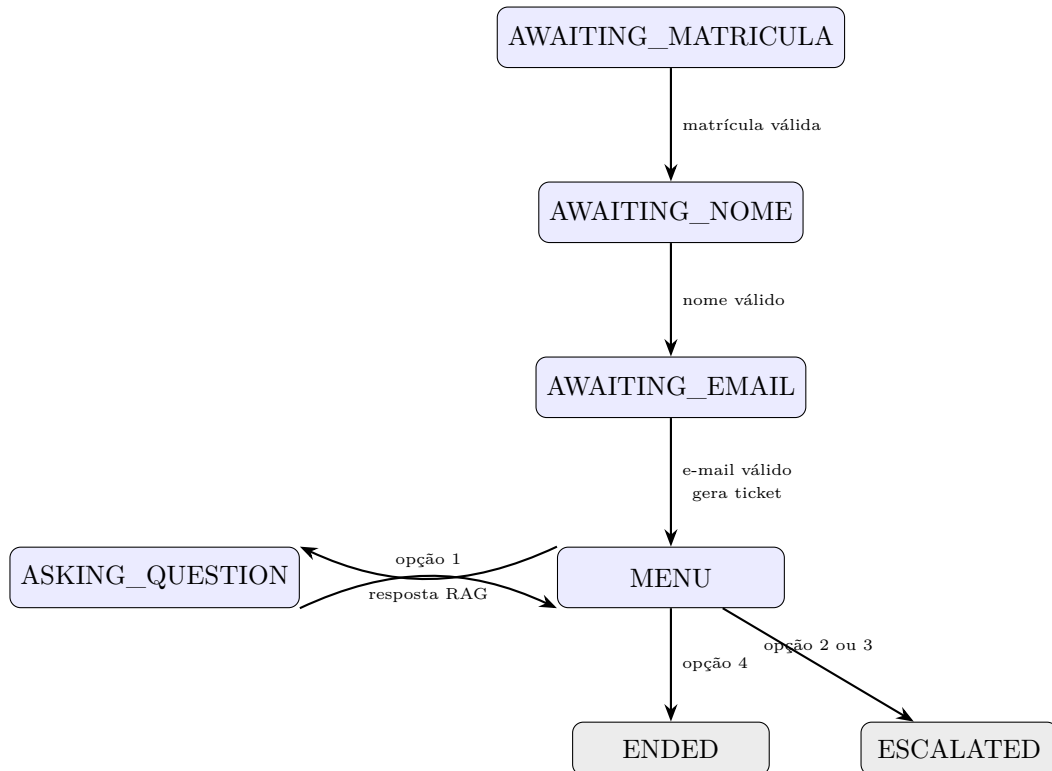


Figura 3.3: Máquina de estados finita do Diretor Virtual (**ChatState**).

O estado **ASKING_QUESTION** delega ao pipeline RAG apresentado no capítulo anterior (Equação (2.1) e seguintes) e retorna a **MENU** após responder, permitindo múltiplas perguntas na mesma sessão. Os estados **ESCALATED** e **ENDED** são absorventes: uma vez atingidos, o *webhook* para de processar novas mensagens daquela conversa (Seção 3.7), devolvendo o controle ao atendente humano ou encerrando o atendimento.

3.6 Endpoints Principais

A Tabela 3.2 resume os *endpoints* relevantes para a arquitetura omnichannel, todos expostos pelo backend FastAPI sob o prefixo `/api/v1`.

Método/Rota	Consumidor	Descrição
POST /chatbot/webhook/chatwoot	Chatwoot (<i>webhook</i> de conta)	Ponto único de entrada omnichannel; roda a máquina de estados e publica a resposta de volta no Chatwoot.
POST /chatbot/start	Clientes REST legados (ex.: integrações futuras)	Cria uma sessão endereçada por <code>session_id</code> e retorna a saudação.
POST /chatbot/message	Clientes REST legados	Processa uma mensagem em uma sessão <code>session_id</code> existente.
GET /chatbot/chatwoot-check	Operação / diagnóstico	Testa a conectividade com o Chatwoot e lista <i>inboxes</i> e times configurados.
GET /chatbot/health	<i>Health check</i>	Disponibilidade do serviço de chatbot.
POST /diretor-virtual/chat	Página /diretor-virtual (RAG avulso)	Consulta direta ao RAG, sem máquina de estados nem integração Chatwoot; mantido para o caso de uso de pergunta única fora do fluxo de ticket.

Tabela 3.2: Endpoints do backend FastAPI relevantes à arquitetura omnichannel.

3.7 Escalação para Atendimento Humano

Ao escolher “Falar com Secretário” ou “Falar com Diretor” no menu, a sessão é encaminhada à equipe (*team*) correspondente no Chatwoot. A lógica de escalação bifurca conforme a origem da sessão:

- **Sessão originada do *webhook*** (widget web ou WhatsApp) — já existe uma *conversation* no Chatwoot (a mesma que disparou o *webhook*). A escalação apenas *atribui a equipe* a essa conversa e publica uma nota interna com o resumo do atendimento, preservando o histórico já em andamento.
- **Sessão originada do fluxo REST legado** — não há conversa prévia no Chatwoot; a escalação cria um contato e uma conversa novos.

```
1 if session.chatwoot_conversation_id:
```



```

2      # Conversa ja existe (webhook): apenas atribui e avisa nela
3      assign_team(session.chatwoot_conversation_id, team_id)
4      send_message(session.chatwoot_conversation_id, context,
5                    private=True)
6      conv_id = session.chatwoot_conversation_id
7  else:
8      # Sem conversa previa (REST legado): cria contato + conversa
9      conv_id = escalate_to_team(name=session.nome, email=session.
10                                email,
11                                matricula=session.matricula,
12                                ticket_id=session.ticket_id,
13                                team_id=team_id,
14                                inbox_id=settings.
15                                    chatwoot_inbox_id_chatweb,
16                                context_message=context)

```

Listing 3.3: Bifurcação de escalação (chatbot_service.py).

Por que não sempre criar uma conversa nova?

Criar uma segunda conversa para uma sessão que já chegou via *webhook* duplicaria o atendimento no painel do Chatwoot — o atendente veria duas conversas para o mesmo usuário: uma vazia (a nova) e outra com o histórico real (a original). Reutilizar `chatwoot_conversation_id` quando ele já existe preserva a continuidade do atendimento.

3.8 Extensibilidade Multi-Canal: o Caso do WhatsApp

Como o *webhook* do Chatwoot é assinado por conta, habilitar o bot no WhatsApp não exigiu nenhuma configuração adicional no Chatwoot — apenas incluir o `inbox_id` do canal WhatsApp na *allowlist* do backend:

```

1 chatwoot_inbox_id_chatweb: int = Field(default=5)
2 chatwoot_inbox_id_whatsapp: int = Field(default=0)
3
4 @property
5 def chatwoot_bot_inbox_ids(self) -> set[int]:
6     ids = {self.chatwoot_inbox_id_chatweb}
7     if self.chatwoot_inbox_id_whatsapp:
8         ids.add(self.chatwoot_inbox_id_whatsapp)
9     return ids

```

Listing 3.4: *Allowlist* de inboxes (`settings.py`).**Estratégia Anterior (preservada para benchmarking)**

Antes da integração via *webhook*, o fluxo de WhatsApp — orquestrado por uma automação n8n (N8N.IO, 2019) (*fair-code*), que já consumia *webhooks* do Chatwoot para outras finalidades — chamava diretamente o *endpoint* `/diretor-virtual/chat`: RAG puro, sem máquina de estados, sem coleta de matrícula/e-mail e sem geração de ticket.

Problema Observado

Como o Chatwoot dispara `message_created` para *todos* os *webhooks* da conta, habilitar o bot no inbox do WhatsApp fazia com que **duas** respostas concorrentes fossem geradas para a mesma mensagem: a automação n8n (RAG puro, fluxo antigo) e o novo *webhook* do backend (máquina de estados completa). O usuário do WhatsApp recebia respostas duplicadas e divergentes.

A resolução foi arquitetural, não apenas de código: a etapa do fluxo n8n que chamava `/diretor-virtual/chat` e publicava a resposta foi desativada, deixando o *webhook* do backend como única fonte de resposta automática para o inbox do WhatsApp — o n8n permanece disponível para outras automações que não conflitem com o fluxo de atendimento.

3.9 Considerações de Segurança

- **Prevenção de laço** — o filtro `message_type == incoming` (Seção anterior) é a defesa primária contra o bot reprocessar suas próprias respostas.
- **Escopo por *inbox*** — a *allowlist* `chatwoot_bot_inbox_ids` garante que canais fora do escopo do bot (ex.: e-mail institucional) nunca acionam a máquina de estados, mesmo compartilhando a mesma conta Chatwoot.
- **Segredo do *webhook*** — o Chatwoot permite associar um segredo compartilhado a cada *webhook*. A verificação dessa assinatura no backend ainda não está implementada e é registrada como trabalho futuro (item 8).
- **Menor privilégio da API** — o *token* de API do Chatwoot usado pelo backend (`CHATWOOT_API_TOKEN`) concede acesso à conta inteira; um *agent bot* dedicado, com escopo restrito às *inboxes* do bot, é uma evolução natural desse desenho.

Os diagramas desta seção são produção original do autor, licenciados © (CC BY 4.0).

Capítulo 4

Módulo ACC — Extração Inteligente de Documentos

4.1 Contexto e Desafio

As Atividades Complementares de Graduação (ACC/ACG) são requisito curricular para integralização do curso de Sistemas de Informação. Historicamente, a validação dessas atividades é feita manualmente por servidores, que examinam cada certificado entregue pelo aluno e somam as cargas horárias.

O processo é:

- **Heterogêneo** — certificados variam em formato, layout, idioma e fonte.
- **Propenso a erros** — leitura manual de dezenas de certificados por aluno.
- **Não escalável** — custo cresce linearmente com o número de alunos.

O módulo ACC do FasiTech automatiza a extração da carga horária via *Document AI* com Modelos de Linguagem Multimodais.

4.2 Fundamentos: Document AI e Multimodal LLMs

Document AI é o campo de estudo da compreensão automática de documentos não-estruturados (CUI et al., 2021). Diferentemente de NLP clássico — que opera sobre texto puro —, Document AI lida com a informação visual e estrutural de documentos (layout, tabelas, fontes, posição do texto).

4.2.1 Modelos Multimodais

Modelos de linguagem multimodais (*Multimodal LLMs*) processam imagens e texto de forma conjunta, sem necessidade de OCR separado. O **Gemini** (GEMINI TEAM et al.,

2023) é um modelo multimodal nativo que interpreta texto, imagens e PDFs em uma única passagem, sendo especialmente adequado para certificados escaneados.

A geração de resposta y dado um documento D (página PDF) e uma instrução I pode ser expressa como:

$$y = \arg \max_y p_\theta(y \mid D, I) \quad (4.1)$$

onde D é representado como sequência de tokens visuais extraídos por um *Vision Encoder* e I é o prompt de instrução.

4.2.2 Processamento de PDFs com pdf2image e Poppler

Quando o modelo não suporta PDF nativo, a conversão para imagens de alta resolução (300 DPI) via *pdf2image/Poppler* é prática padrão (BLECHER et al., 2023). O FasiTech oferece suporte a ambos os modos:

1. **PDF direto** — enviado como objeto **File** ao Gemini, que processa nativamente todas as páginas.
2. **PDF** $\rightarrow$ **imagem** — cada página é convertida a PNG 300 DPI e enviada individualmente ao modelo via **Image**.

4.3 Arquitetura do Módulo ACC

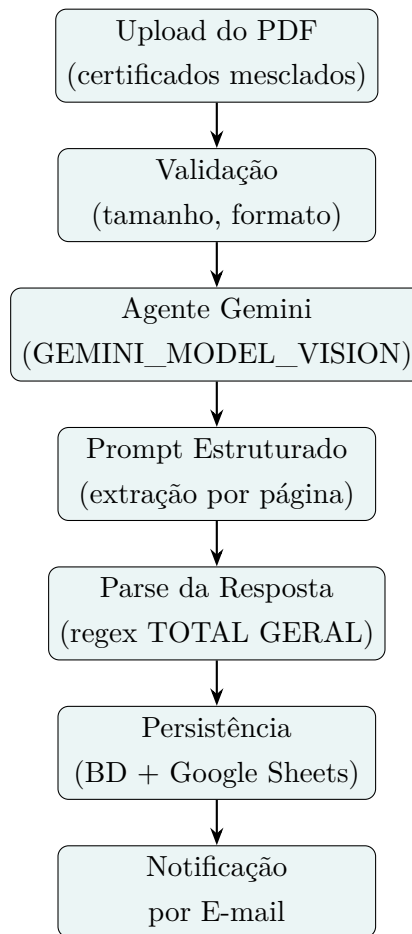


Figura 4.1: Pipeline de processamento do Módulo ACC.

4.4 Prompt Engineering para Extração Estruturada

O desempenho de LLMs em tarefas de extração de informação depende criticamente da qualidade do prompt (BROWN et al., 2020). O FasiTech utiliza um prompt de *few-shot* implícito com especificação de:

- **Papel e tarefa** — “Você é um especialista em extrair informações de certificados acadêmicos”.
- **Formatos esperados** — lista de padrões comuns (“40h”, “40 horas”, “CH: 40h”, etc.).
- **Formato de saída** — estrutura canônica por página e linha TOTAL GERAL: X horas.
- **Tratamento de casos-limite** — carga horária em dias (8h/dia), múltiplas cargas na página, ausência de carga.

4.4.1 Extração do Total por Expressão Regular

Após a resposta do modelo, o sistema executa parsing determinístico para extrair o total numérico, mitigando alucinações:

```

1 padroes = [
2     r'TOTAL GERAL:\s*(\d+(?:,\d+)?)\s*horas?',
3     r'Total Geral:\s*(\d+(?:,\d+)?)\s*horas?',
4     r'TOTAL:\s*(\d+(?:,\d+)?)\s*horas?',
5 ]
6 for padrao in padroes:
7     match = re.search(padrao, texto, re.IGNORECASE)
8     if match:
9         horas = match.group(1).replace(',', '.', '')
10        return f"TOTAL GERAL: {horas} horas"

```

Listing 4.1: Extração de total com regex.

4.5 Análise de Confiabilidade e Limitações

4.5.1 Fontes de Erro

Tabela 4.1: Fontes de erro e mitigações no módulo ACC.

Fonte	Descrição	Mitigação
Baixa resolução	PDFs escaneados em < 150 DPI	Reprocessamento a 300 DPI
Tabelas complexas	Layout de certificado não padrão	Prompt com múltiplos formatos
Alucinação do modelo	LLM inventa carga não presente	Regex pós-geração + fallback manual
Idioma estrangeiro	Certificados em inglês/espanhol	Instrução multilíngue no prompt

4.5.2 Escalabilidade

O modelo é chamado uma vez por envio (PDF completo), tornando o custo fixo por aluno independente do número de páginas dentro do limite de contexto do Gemini 2.5 Flash Lite ($\approx 1,5 \times 10^6$ tokens). Para envios com muitos certificados, o sistema pode ser estendido com paginação em lotes de 10–15 páginas por chamada.

Capítulo 5

Infraestrutura e Integração

5.1 Banco de Dados Vetorial: LanceDB

LanceDB (LANCEDB, 2023) é um banco de dados vetorial embutido (*embedded*) baseado no formato Apache Lance, projetado para alto desempenho em hardware de commodity. No FasiTech, ele persiste:

- Tabela `recipes` — chunks dos documentos com vetores 768-d.
- Tabela `semantic_cache` — entradas de cache semântico com vetores, metadados e timestamps.

5.2 Modelos de Linguagem e Embedder

O FasiTech utiliza o framework **Agno** (ex-Phidata) para abstrair a camada de LLM. A seleção do modelo de geração é feita em tempo de inicialização do `ChatbotService`:

1. Se `GOOGLE_API_KEY` estiver configurada: usa **Gemini** via API Google.
2. Caso contrário: usa **Ollama** (OLLAMA, 2023) local como fallback (modelo `OLLAMA_LLM_MODEL`).

Tabela 5.1: Configuração dos modelos no FasiTech (`LLMConfig.py`).

Variável	Uso	Padrão
<code>GEMINI_MODEL</code>	Geração de respostas — Diretor Virtual	<code>gemini-2.5-flash</code>
<code>GEMINI_MODEL_VISION</code>	Extração de certificados — ACC	<code>gemini-2.5-flash-lite</code>
<code>OLLAMA_LLM_MODEL</code>	Fallback local (sem API key)	<code>qwen2.5:3b</code>
<code>nomic-embed-text</code>	Embeddings 768-d (sempre Ollama)	<code>OllamaEmbedder</code>

Nota: O embedder (`nomic-embed-text`) é sempre local via Ollama, independentemente da disponibilidade da API Google. O modelo de geração (Gemini vs. Ollama) é o único componente que muda conforme o ambiente.

5.3 Padrão de Projeto: Singleton com Inicialização Lazy

O `ChatbotService` implementa o padrão Singleton via `__new__`, garantindo que a inicialização custosa (conexão com Ollama, criação do índice LanceDB, carregamento dos documentos) ocorra apenas uma vez por processo. O diretório de dados é `~/.cache/fasitech/rag/` (persistência por usuário do SO, compatível com ambientes Docker).

Sessões de usuário são isoladas por `session_id`; cada sessão cria um `Agent Agno` sob demanda com histórico de conversa independente armazenado em SQLite (`.db` no mesmo diretório de cache). Quando `persist_history=False`, o histórico fica apenas em RAM.

Capítulo 6

Histórico de Estratégias e Benchmarking

Este capítulo consolida a evolução das estratégias de recuperação do Diretor Virtual. As versões anteriores são preservadas intencionalmente — tanto para rastreabilidade quanto para servir de *baseline* em *benchmarks* futuros. Para cada componente, registra-se a estratégia anterior, o problema observado e a estratégia atual.

Tabela 6.1: Evolução das estratégias de recuperação (RAG) do FasiTech.

Componente	Estratégia anterior (baseline)	Problema observado	Ref.
Chunking	Cabeçalho corrente sobrescrito por qualquer heading; “Ementa:” virava a seção do chunk	Nome da disciplina descartado; ementas curtas (ED II) não-recuperáveis	2.2.3
Embedding	<code>topic-embed-text</code> sem prefixo de tarefa (mesmo texto p/ query e doc)	Recuperação assimétrica degradada; queda de <i>recall</i>	2.1.1
Fusão híbrida	<i>Insert-at-front</i> : keyword inserido no topo, sem RRF real; <i>pool</i> pequeno	Dominância de um recuperador; acertos só-esparsos (ED II, pos. 21) cortados	2.1.2
Reordenação	Nenhuma (ordem da recuperação ia direto ao prompt)	Similaridade $\neq$ relevância; sem desambiguar “I” vs. “II”	2.7

Caso de regressão de referência (*golden case*). A consulta “Qual a ementa da disciplina Estrutura de Dados II?” é o caso canônico de teste: antes das melhorias retornava “não encontrei”; após *chunking com breadcrumb* (Seção 2.2.3) + *prefixos topic* (Seção 2.1.1) + *RRF com pool ampliado* (Seção 2.1.2) + *reranking* (Seção 2.7), o chunk correto passa a ocupar a posição 1 e a resposta cita a ementa real (“Árvores... Grafos... Tabela de Hash”). Recomenda-se manter este e os demais pares disciplina/ementa I/II como conjunto

mínimo de regressão.

Reprodutibilidade do índice

Mudanças de *chunking* ou *embedding* não alteram os arquivos fonte e, portanto, não eram detectadas pelo *hash* de documentos. Introduziu-se a constante `INDEX_PIPELINE_VERSION`, incorporada ao *hash*: ao incrementá-la, força-se reindexação limpa e a limpeza do cache semântico, garantindo que cada *benchmark* rode sobre um índice consistente com o código.

Capítulo 7

Gestão de Funcionários e Roteamento Dinâmico de Notificações

Ponto de Controle Temporal

Data da mudança: 10 de junho de 2026.

Escopo: introdução do cadastro de funcionários como fonte única de verdade para (i) os seletores de orientadores, docentes, pareceristas e membros de banca nos formulários e (ii) o roteamento de notificações por e-mail.

Motivação: eliminar a digitação manual de nomes e a configuração de destinatários em variáveis de ambiente (`.env`), substituindo-as por dados governados em banco de dados.

Componentes afetados: tabela `funcionarios`; endpoint público `/api/v1/funcionarios`; formulários de TCC, Estágio, Requerimento de TCC, Plano de Ensino, Projetos e ACC; agendador de alertas acadêmicos.

7.1 Contexto e Motivação

Antes desta mudança, os formulários acadêmicos do FasiTech apresentavam o campo de orientador (e correlatos) como *texto livre* ou listas *hard-coded* no *frontend*, e os destinatários das notificações por e-mail eram lidos de variáveis de ambiente (`TCC_RECIPIENTS`, `DESTINATARIOS`, `PARECERISTAS` etc.). Esse arranjo apresentava três limitações:

1. **Inconsistência de dados** — o mesmo docente podia ser grafado de formas distintas em submissões diferentes, dificultando consultas e cruzamentos.
2. **Acoplamento à infraestrutura** — alterar um destinatário exigia editar o `.env` e reimplantar a aplicação.
3. **Ausência de papéis** — não havia forma de expressar que um funcionário exerce

uma função institucional (Diretor, Coordenador de Estágio) para fins de roteamento automático.

Banco de dados como fonte única de verdade

Centralizar pessoas e papéis em uma entidade persistente (**funcionarios**) desacopla regra de negócio de configuração de ambiente. Seletores e destinatários passam a derivar de consultas, tornando a manutenção uma operação de dados (CRUD na interface administrativa) e não de reimplantação.

7.2 Modelo de Dados: a Entidade Funcionario

A entidade **Funcionario** registra docentes e colaboradores da FASI. Além dos atributos descritivos, três *flags* booleanas modelam funções institucionais, e os campos categóricos **tipo** e **categoria** permitem segmentar a população para os seletores.

Campo	Domínio	Descrição
nome	texto	Nome completo (chave de casamento).
filiacao	texto	Filiação (opcional).
titulo	{Doutorado, Mestrado, Especialista, Graduação}	Titulação acadêmica.
tipo	{Interno, Externo}	Vínculo com a faculdade.
categoria	{Docente, Secretaria, Colaborador}	Categoria funcional.
email	texto	Endereço para notificações.
fone	texto	Contato (WhatsApp).
data_aniversario	data	Data de aniversário.
diretor_faculdade	booleano	Marca o Diretor da Faculdade.
coordenador_estagio	booleano	Marca o Coordenador de Estágio.
representante_docente	booleano	Marca o Representante Docente.

Tabela 7.1: Atributos da entidade **Funcionario**.

O gerenciamento (CRUD) é exposto na seção *Configuração* da interface administrativa, sob autenticação por *token*. A criação de colunas em bases já existentes é idempotente: a inicialização aplica `ALTER TABLE` condicional para os campos novos (categoria e *flags* de cargo), preservando registros legados com valores-padrão.

7.3 Seletores Dinâmicos nos Formulários

Um *endpoint* público de leitura, `GET /api/v1/funcionarios`, fornece a lista de nomes para os seletores. Por privacidade, ele expõe apenas dados não sensíveis (`nome`, `titulo`,

tipo, categoria) — **nunca** e-mail ou telefone. A filtragem por papel ocorre no cliente, permitindo combinações lógicas. A Tabela 7.2 resume as regras de elegibilidade adotadas em cada formulário.

Formulário / campo	Conjunto elegível (predicado sobre funcionarios)
TCC — Orientador	$\text{categoria} = \text{Docente} \wedge \text{tipo} = \text{Interno}$
Estágio — Orientador/Supervisor	$\text{categoria} = \text{Docente} \vee \text{categoria} = \text{Colaborador}$
Requerimento TCC — Orientador	$\text{categoria} = \text{Docente} \wedge \text{tipo} = \text{Interno}$
Requerimento TCC — Banca	$\text{categoria} = \text{Docente} (\text{Interno} \vee \text{Externo})$
Plano de Ensino — Docente	$\text{categoria} = \text{Docente} (\text{Interno} \vee \text{Externo})$
Projetos — Docente e Pareceristas	$\text{categoria} = \text{Docente} \wedge \text{tipo} = \text{Interno}$

Tabela 7.2: Regras de elegibilidade dos seletores derivadas do banco.

Tratamento de ausência no cadastro

Quando o nome desejado não consta da lista, a interface orienta o solicitante a requisitar o cadastro à FASI por e-mail, informando *Nome, Filiação, e-mail, Telefone e Titulação*. Assim, a lacuna de dados é resolvida na fonte (o cadastro) em vez de reintroduzir texto livre, preservando a integridade referencial entre submissões e funcionários.

7.4 Roteamento de Notificações por Papel

As notificações de submissão deixaram de usar destinatários do `.env`. O roteamento passa a combinar três fontes derivadas do banco: (i) *funcionários selecionados* no próprio formulário (resolvidos por nome para o respectivo e-mail); (ii) *cargos* (consultados pelas *flags* `diretor_faculdade` e `coordenador_estagio`); e (iii) *categorias* (ex.: **Secretaria** para o “secretário”). Os e-mails de aluno e da biblioteca, quando aplicáveis, são acrescentados explicitamente. A união dos conjuntos é deduplicada de forma *case-insensitive*.

Formulário	Destinatários
Projetos	Docente (coordenador do projeto) + Pareceristas 1 e 2 + Diretor
ACC	Diretor + Secretário + Aluno
Plano de Ensino	Docente (professor interessado) + Diretor
TCC	Orientador + Aluno + Secretário + Biblioteca (<code>bibcameta@ufpa.br</code>)
Requerimento TCC	Orientador + Diretor + Aluno
Estágio	Orientador + Aluno + Coordenador de Estágio + Diretor

Tabela 7.3: Política de roteamento de notificações por formulário.

Adicionalmente, o agendador de alertas acadêmicos, cujo destino “docentes” antes

lia DESTINATARIOS/PARECERISTAS do .env, passou a consultar os **docentes internos** (`categoria = Docente ^ tipo = Interno`) diretamente do banco.

Composição e deduplicação de destinatários

O roteamento é expresso como a união de conjuntos $R = S \cup C \cup K \cup \{e_{\text{aluno}}, e_{\text{biblioteca}}\}$, onde S são os selecionados no formulário, C os cargos e K as categorias relevantes. A deduplicação normaliza por caixa baixa, de modo que um docente que também seja Diretor (ou Orientador que seja Coordenador) recebe uma única cópia da notificação.

Capítulo 8

Geração Automática de Ata de Defesa de TCC

Ponto de Controle Temporal

Data da implementação: 10 de junho de 2026.

Escopo: geração automatizada de Ata de Defesa em formato `.doc` a partir dos dados do Requerimento de TCC, com envio por e-mail ao orientador acompanhado de instruções para cadastro no SIPAC.

Motivação: eliminar a produção manual de atas, reduzindo erros de digitação e acelerando o trâmite burocrático pós-defesa.

Componentes afetados: módulo `backend/infrastructure/documents/ata_defesa.py`; endpoint `POST /api/v1/requerimento-tcc/{id}/gerar-ata`; painel *Consulta Requerimento TCC* (botão “Gerar ATA”).

8.1 Contexto e Motivação

Após a defesa do TCC, o orientador deve produzir a **Ata de Defesa** para registro oficial no SIPAC. Até esta implementação, o documento era criado manualmente pelo orientador, copiando dados do aluno, da banca e da defesa em um modelo Word institucional. Esse processo apresentava três pontos de atrito:

1. **Retrabalho de dados** — informações já submetidas pelo aluno no *Requerimento de TCC* (título, banca, data/hora da defesa) tinham que ser redigitadas na ata.
2. **Inconsistência de formato** — a ausência de validação permitia erros de ortografia, omissão de membros ou formatação divergente do modelo oficial.
3. **Latência administrativa** — o atraso na produção da ata postergava o registro no SIPAC, bloqueando a homologação.

Geração a partir da fonte de verdade

O *Requerimento de TCC* é submetido **antes** da defesa e contém todos os dados necessários para a ata: aluno, orientador, banca, título, data/hora, local. Gerar a ata a partir dessa fonte elimina retrabalho e garante correspondência exata entre o registro de intenção de defesa e o documento de homologação.

8.2 Arquitetura da Solução

A geração da ata é implementada como um *pipeline* de três estágios: (1) recuperação dos dados no banco, (2) composição do documento .docx com a biblioteca `python-docx`, e (3) envio por e-mail ao orientador com o documento anexado e instruções textuais para cadastro no SIPAC.

8.2.1 Recuperação de Dados Enriquecidos

O *endpoint* POST `/api/v1/requerimento-tcc/{id}/gerar-ata` recebe o identificador da submissão e recupera o registro completo da tabela `requerimento_tcc_submissions`, cruzando-o com a tabela `funcionarios` para obter a **titulação** e o **tipo** (Interno/Externo) de cada membro da banca. Esse cruzamento é feito por *casamento de nome*, normalizado para minúsculas e sem acentuação. A lógica de resolução:

```

1 func_map = {f["nome"].strip().lower(): f for f in funcionarios}
2
3 def _get_func(nome: str | None) -> dict:
4     if not nome:
5         return {}
6     return func_map.get(nome.strip().lower(), {})
7
8 ori_f = _get_func(orientador_nome)
9 m1_f = _get_func(membro_banca1_nome)
10 m2_f = _get_func(membro_banca2_nome)
11 m3_f = _get_func(membro_banca3_nome) if membro_banca3_nome else
    {}

```

Listing 8.1: Busca de titulação e tipo por nome de funcionário.

Cada `dict` retornado contém `titulo` e `tipo`. O `titulo` é convertido para abreviação (“Doutorado” → “Dr.”, “Mestrado” → “M.Sc.”, “Especialista” → “Esp.”, “Graduação” → sem abreviação), e o `tipo` determina a filiação exibida na assinatura (“Interno” → “FASI”, “Externo” → “Externo”).

8.2.2 Composição do Documento com `python-docx`

A ata segue o modelo oficial da UFPA, estruturado como:

1. **Cabeçalho institucional:** brasão da UFPA (PNG), seguido das linhas centralizadas “SERVIÇO PÚBLICO FEDERAL / UNIVERSIDADE FEDERAL DO PARÁ / CAMPUS UNIVERSITÁRIO DO TOCANTINS - CAMETÁ / FACULDADE DE SISTEMAS DE INFORMAÇÃO – FASI”.
2. **Título:** “ATA DE DEFESA DE TRABALHO DE CONCLUSÃO DE CURSO” (negrito, centralizado).
3. **Corpo:** parágrafo justificado iniciado por “Aos [dia por extenso] ([dia numérico]) dias do mês de [mês por extenso] de [ano por extenso] ([ano numérico]), às [hora por extenso] ([hora numérica]h) reuniu-se via [local da defesa]...”, incluindo composição da banca, título do trabalho (em negrito), nome e matrícula do discente (em maiúsculas e negrito), e os campos de resultado e conceito em **vermelho e negrito**: [APROVADO|REPROVADO] e [INSUFICIENTE|REGULAR|BOM|EXCELENTE].
4. **Bloco de assinaturas:** uma linha centralizada para cada membro da banca (“Prof. [abrev.] [nome] / [papel] – [filiação]”) e o aluno.

A biblioteca `python-docx` permite controle fino sobre formatação de parágrafos (alinhamento, espaçamento before/after) e *runs* (negrito, cor RGB, fonte). O documento é composto em memória e serializado para `bytes` com `doc.save(io.BytesIO())`.

Campos editáveis destacados em vermelho

Os campos que o orientador deve preencher manualmente — [APROVADO|REPROVADO] e [INSUFICIENTE|REGULAR|BOM|EXCELENTE] — são renderizados em **vermelho e negrito**, orientando visualmente onde a edição é necessária antes da assinatura.

8.2.3 Normalização de Nomes no Arquivo

O nome do arquivo anexado é `ATA_DEFESA_<NOME_DO_ALUNO>.doc`, onde `<NOME_DO_ALUNO>` é convertido para maiúsculas, **sem acentos** (via `unicodedata.normalize('NFKD')`) e com espaços substituídos por *underscores*. Essa normalização garante compatibilidade com clientes de e-mail e sistemas de arquivos que rejeitam caracteres não-ASCII em nomes de anexo. Exemplo:

Diego Corrêa de Moraes → `ATA_DEFESA_DIEGO_CORREA_DE_MORAES.doc`

8.2.4 Envio por E-mail com Instruções SIPAC

Após a geração do `.docx`, o sistema consulta o e-mail do orientador na tabela `funcionarios` (via `get_funcionario_emails_by_nomes([orientador_nome])`) e envia o documento como anexo. O corpo do e-mail contém um **roteiro passo a passo** para cadastro da ata no SIPAC:

1. Entrar no SIPAC.
2. Cadastrar Documento.
3. Tipo de Documento: “ATA DE DEFESA DE TRABALHO DE CONCLUSÃO DE CURSO (TCC)”.
4. Natureza: “Ostensivo”.
5. Assunto Detalhado: “ATA DE DEFESA DE TRABALHO DE CONCLUSÃO DE CURSO DO ALUNO [NOME]”.
6. Forma de Documento: “Anexar documento”.
7. Marcar os envolvidos (UFPA ou externos via `sou.gov` / SIGAA).
8. Informar orientador e aluno como interessados.
9. Unidade de Destino: “11.19.36” (código da FASI).
10. Conferir e finalizar.

Essa abordagem elimina a necessidade de documentação externa ou treinamento presencial sobre o fluxo do SIPAC, pois as instruções são entregues **contextualmente** no momento em que o orientador precisa delas.

8.3 Interface de Usuário

No painel **Consulta Requerimento TCC** (`/consulta/requerimento-tcc`), cada linha da tabela exibe um botão “**Gerar ATA**” com ícone de documento. Ao clicar:

1. O botão entra em estado de *loading* (“Gerando...”).
2. O *frontend* faz uma requisição `POST /api/v1/requerimento-tcc/{id}/gerar-ata`.
3. O *backend* gera a ata e envia o e-mail.
4. Um *toast* de sucesso exibe a mensagem “ATA gerada e enviada ao orientador ([e-mail])”, ou um erro caso o orientador não tenha e-mail cadastrado.

Essa integração de um único clique reduz a latência entre a realização da defesa e a disponibilização do documento para registro oficial de **horas** para **segundos**.

Geração sob demanda vs. automática

A geração da ata é disparada **manualmente** pela coordenação (via botão no painel administrativo), e não automaticamente ao final da defesa. Isso permite que eventuais correções de última hora nos dados da banca ou do título sejam feitas antes da geração do documento oficial. Uma vez gerada, a ata pode ser regerada quantas vezes necessário (o sistema não impede múltiplos disparos para o mesmo requerimento).

8.4 Tratamento de Erros e Validações

O pipeline contempla três pontos de falha e seus respectivos tratamentos:

1. **Requerimento não encontrado** — retorna HTTP 404 com a mensagem “Requerimento não encontrado”.
2. **E-mail do orientador ausente** — lança `ValueError("E-mail do orientador '[nome]' não encontrado na base de dados.")` convertida para HTTP 422. A resposta orienta o operador a cadastrar o e-mail no painel de Funcionários antes de tentar novamente.
3. **Falha no envio do e-mail** — o serviço de e-mail (`send_email_with_attachments`) captura exceções do SMTP e registra erro no log, mas **não bloqueia** a resposta da API (a ata foi gerada com sucesso; apenas o envio falhou).

Capítulo 9

Trabalhos Futuros

1. **Multi-Query Retrieval** — gerar 3 variações da pergunta via LLM antes da busca e fundir resultados com RRF (ES et al., 2023; LANGCHAIN, 2023).
2. **HyDE** (*Hypothetical Document Embeddings*) — embeddar uma resposta hipotética gerada pelo LLM em vez da query original para maior precisão semântica (GAO et al., 2023).
3. **Parent-Child Chunking** — indexar chunks pequenos para busca, retornar chunk pai para geração (LLAMAINDEX, 2023).
4. **Migração de Embedding para bge-m3** — modelo multilíngue (denso + esparso) com melhor desempenho em português segundo o MTEB; avaliar empiricamente contra o `nomic-embed-text` com prefixos via *harness*.
5. **Harness de Avaliação** — conjunto de perguntas-gabarito (ex.: todas as ementas I/II) com métricas *hit-rate*/MRR para medir regressões a cada mudança de pipeline.
6. **Interface de Feedback** — expor botões de avaliação (1–5 estrelas) no ChatWidget para alimentar o cache semântico com dados reais de usuários.
7. **Validação Cruzada ACC** — comparar resultado do Gemini com extração por OCR clássico (Tesseract) para medir concordância e detectar certificados problemáticos.
8. **Verificação de assinatura do *webhook*** — validar, no `endpoint /chatbot/webhook/chatwoot`, o segredo compartilhado (HMAC) enviado pelo Chatwoot em cada requisição, rejeitando payloads cuja assinatura não confira.
9. **Agent Bot dedicado** — migrar do *token* de API de conta inteira para um *agent bot* do Chatwoot com escopo restrito às *inboxes* do bot conversacional.

Referências

- ALIANNEJADI, Mohammad et al. ClariQ: Building a Dataset for Clarification of Questions in Open-Domain Information-Seeking Conversations. In: PROCEEDINGS of the 59th Annual Meeting of the Association for Computational Linguistics (ACL) Findings. [S.l.: s.n.], 2021. Disponível em: <<https://arxiv.org/abs/2009.11352>>.
- ANTHROPIC. **Introducing Contextual Retrieval**. [S.l.: s.n.], 2024. Blog post. Disponível em: <<https://www.anthropic.com/news/contextual-retrieval>>.
- BANG, Yue et al. **GPTCache: A Data Store for Efficient LLM Responses**. [S.l.: s.n.], 2023. <https://github.com/zilliztech/GPTCache>.
- BLECHER, Lukas et al. Nougat: Neural Optical Understanding for Academic Documents. In: ARXIV preprint. [S.l.: s.n.], 2023. Disponível em: <<https://arxiv.org/abs/2308.13418>>.
- BROWN, Tom B. et al. Language Models are Few-Shot Learners. **Advances in Neural Information Processing Systems (NeurIPS)**, v. 33, p. 1877–1901, 2020. Disponível em: <<https://arxiv.org/abs/2005.14165>>.
- CARPINETO, Claudio; ROMANO, Giovanni. A Survey of Automatic Query Expansion in Information Retrieval. **ACM Computing Surveys**, v. 44, n. 1, p. 1–50, 2012.
- CHATWOOT. **Chatwoot: Open-source customer engagement suite**. [S.l.: s.n.], 2023. <https://www.chatwoot.com>.
- CORMACK, Gordon V.; CLARKE, Charles L. A.; BUETTCHER, Stefan. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In: PROCEEDINGS of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval. [S.l.: s.n.], 2009. P. 758–759.
- CUI, Lei et al. Document AI: Benchmarks, Models and Applications. **arXiv preprint**, 2021. Disponível em: <<https://arxiv.org/abs/2111.08609>>.
- ES, Shahul et al. **RAGAS: Automated Evaluation of Retrieval Augmented Generation**. [S.l.: s.n.], 2023. <https://arxiv.org/abs/2309.15217>.

GAO, Luyu et al. Precise Zero-Shot Dense Retrieval without Relevance Labels. In: PROCEEDINGS of the 61st Annual Meeting of the Association for Computational Linguistics (ACL). [S.l.: s.n.], 2023. Disponível em:

<<https://arxiv.org/abs/2212.10496>>.

GEMINI TEAM et al. Gemini: A Family of Highly Capable Multimodal Models. **arXiv preprint**, 2023. Disponível em: <<https://arxiv.org/abs/2312.11805>>.

KARPUKHIN, Vladimir et al. Dense Passage Retrieval for Open-Domain Question Answering. In: PROCEEDINGS of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). [S.l.: s.n.], 2020. P. 6769–6781. Disponível em:

<<https://arxiv.org/abs/2004.04906>>.

KHATTAB, Omar; ZAHARIA, Matei. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In: PROCEEDINGS of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval. [S.l.: s.n.], 2020. P. 39–48. Disponível em:

<<https://arxiv.org/abs/2004.12832>>.

LANCEDB. **LanceDB: Developer-friendly, serverless vector database**. [S.l.: s.n.], 2023. <https://lancedb.github.io/lancedb/>.

LANGCHAIN. **Multi-Query Retriever**. [S.l.: s.n.], 2023. https://python.langchain.com/docs/modules/data_connection/retrievers/MultiQueryRetriever.

LEWIS, Patrick et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2020. v. 33, p. 9459–9474. Disponível em: <<https://arxiv.org/abs/2005.11401>>.

LLAMAINDEX. **Small-to-Big Retrieval and Parent Document Retriever**. [S.l.: s.n.], 2023. https://docs.llamaindex.ai/en/stable/examples/retrievers/parent_child_retrieval.html.

N8N.IO. **n8n: Fair-code workflow automation platform**. [S.l.: s.n.], 2019. <https://n8n.io>.

NUSSBAUM, Zach et al. **Nomic Embed: Training a Reproducible Long Context Text Embedder**. [S.l.: s.n.], 2024. <https://arxiv.org/abs/2402.01613>.

OLLAMA. **Ollama: Get up and running with large language models locally**. [S.l.: s.n.], 2023. <https://ollama.ai>.

ROBERTSON, Stephen E. et al. Okapi at TREC-3. In: PROCEEDINGS of the Third Text REtrieval Conference (TREC-3). [S.l.: s.n.], 1994.

SHI, Freda et al. Large Language Models Can Be Easily Distracted by Irrelevant Context. In: PROCEEDINGS of the 40th International Conference on Machine Learning (ICML). [S.l.: s.n.], 2023. Disponível em: <<https://arxiv.org/abs/2302.00093>>.

VERHOEF, Peter C.; KANNAN, P. K.; INMAN, J. Jeffrey. From Multi-Channel Retailing to Omni-Channel Retailing: Introduction to the Special Issue on Multi-Channel Retailing. **Journal of Retailing**, v. 91, n. 2, p. 174–181, 2015.

ZHU, Zhuoran et al. Semantic Cache for Large Language Model Inference with Similarity-Based Retrieval. **arXiv preprint**, 2023. Disponível em: <<https://arxiv.org/abs/2309.09558>>.